

Guia 2 — Staging Area e Commits Avançados

Este guia aprofunda o funcionamento da **staging area**, explica como o Git registra mudanças e apresenta comandos essenciais para trabalhar com commits de forma mais avançada.

1. O que é a Staging Area (de verdade)

A *staging area* é uma área intermediária entre:

- os arquivos do seu projeto (working directory)
- o commit que você vai criar (repositório Git)

Ela funciona como uma **bandeja de supermercado**:

- você coloca itens nela (`git add`)
- depois passa tudo no caixa de uma vez (`git commit`)

Isso permite:

- escolher exatamente o que vai entrar no commit
- separar mudanças diferentes em commits diferentes
- evitar salvar coisas por engano
- criar commits limpos e organizados

Você vê o que está na staging area usando:

```
git status
```

Exemplo:

```
Changes to be committed:  
new file:   script.py
```

2. Estados dos arquivos no Git

Um arquivo pode estar em um destes estados:

2.1. `untracked` — não rastreado

```
git status
```

Saída:

```
Untracked files:
  script.py
```

2.2. **modified** — modificado, mas não preparado

```
git status
```

Saída:

```
Changes not staged for commit:
  modified:   app.py
```

2.3. **staged** — preparado para commit

```
git add app.py
git status
```

Saída:

```
Changes to be committed:
  modified:   app.py
```

2.4. **committed** — registrado no histórico

```
git commit -m "Atualiza app"
git status
```

Saída:

```
On branch main
nothing to commit, working tree clean
```

3. Adicionando mudanças de forma avançada

✓ 3.1. Adicionar um arquivo específico

```
git add arquivo.py
```

Saída:

```
Changes to be committed:
  new file:   arquivo.py
```

✓ 3.2. Adicionar todos os arquivos modificados

```
git add .
```

Saída:

```
Changes to be committed:
  modified:   app.py
  new file:   utils.py
  deleted:    old_config.json
```

✓ 3.3. Adicionar apenas parte de um arquivo (interativo)

```
git add -p arquivo.py
```

Saída típica:

```
diff --git a/arquivo.py b/arquivo.py
@@ -1,4 +1,7 @@
 def soma(a, b):
```

```
    return a + b

+def sub(a, b):
+    return a - b

Stage this hunk [y,n,q,a,d,j,J,g,/,e,?]?
```

✓ 3.4. Ver o que está na staging area

```
git diff --staged
```

✓ 3.5. Remover algo da staging area

```
git restore --staged arquivo.py
```

Saída:

```
Changes not staged for commit:
  modified:   arquivo.py
```



4. Commits avançados

✓ 4.1. Editar o último commit (--amend)

```
git commit --amend
```

Saída típica:

```
[main 3f2a1b7] Adiciona função soma e utilidades
 2 files changed, 10 insertions(+)
```

✓ 4.2. Commit com título + descrição

```
git commit -m "Implementa login" -m "Adiciona validação de senha."
```

✓ 4.3. Criar commits vazios

```
git commit --allow-empty -m "Commit vazio"
```



5. Ver diferenças antes de commitar

✓ Working directory (não adicionado)

```
git diff
```

Saída:

```
+def sub(a, b):  
+    return a - b
```

✓ Staging area (será commitado)

```
git diff --staged
```



6. Remover arquivos da staging area

```
git restore --staged arquivo.py
```

Saída:

```
Changes not staged for commit:  
  modified:   arquivo.py
```



7. Desfazendo commits (com segurança)

✓ 7.1. Desfazer commit criando outro (seguro)

```
git revert <hash>
```

Saída:

```
[main 7f8e9d1] Revert "Adiciona função soma"
```

✓ 7.2. Voltar no tempo apagando commits (perigoso)

```
git reset --hard <hash>
```

Saída:

```
HEAD is now at 0f9e8d7 Cria arquivo inicial
```



8. Resumo do fluxo avançado

<code>git status</code>	→ veja o estado
<code>git diff</code>	→ veja o que mudou
<code>git add -p</code>	→ adicione trechos específicos
<code>git commit -m "msg"</code>	→ commit claro
<code>git log --oneline</code>	→ revise o histórico
<code>git revert <hash></code>	→ desfaça commits com segurança



Fim do Guia 2 — Staging Area e Commits Avançados

Agora você entende:

- como funciona a staging area
- como escolher o que entra no commit

- como adicionar trechos específicos (`git add -p`)
- como editar commits (`--amend`)
- como ver diferenças (`git diff`)
- como desfazer mudanças com segurança (`git revert`)
- como evitar reescrever histórico acidentalmente
- como trabalhar com commits de forma profissional

Esse conhecimento é essencial para manter um histórico limpo, organizado e fácil de entender.